arXiv:hep-lat/9811036v2 6 Dec 1998

MDP_QCD: Object Oriented Programming for Lattice Gauge Theory

Make lattice simulations on your home PC

Massimo Di Pierro

Theory Group - Department of Physics and Astronomy

University of Southampton - SO17 1BJ - United Kingdom

Email: mdp@hep.phys.soton.ac.uk

16-10-1998

Abstract

This is a manual (built by examples) to explain the use of MDP_QCD [1]. It consists of an ensemble of classes and functions (written in GNU C++) to help in writing programs for lattice QCD in a particularly Object Oriented fashion. Some tricks are implemented, hidden in the class definition, to optimize speed and reduce memory usage on PCs, workstations and parallel computers with sheared memory.

[1] The code described can be dowloaded from the web page:

www.hep.phys.soton.ac.uk/hepwww/postgrad/M.DiPierro/software.html

It can be freely copied and used as long the name of the author is retained.
The author declines any responsibility for any improper or unauthorized use of this software.
If you find any bug in the code, or you have any suggestion, please report it to the author.

Contents

1 Introduction **3**

2 `MDP_Lib1.h` and its usage **5**
2.1 `class Matrix` . . . 5
2.2 `class Random_generator` . . . 7
2.3 `class JackBoot` . . . 9

3 `MDP_QCD.h` and its usage **10**
3.1 Basic Syntax . . . 10
3.2 Creating gauge configurations . . . 13
3.3 First measurements . . . 16
3.4 Introducing fermions . . . 19
3.5 Stochastic light propagators . . . 22
3.6 Smearing . . . 24
3.7 Examples: fuzzing and cooling . . . 25
3.8 Basic algorithms: `multihit()` . . . 26
3.9 Basic algorithms: `mul_Q()` and `mul_invQ()` . . . 27
3.10 Input/Output . . . 29
3.11 Memory and speed . . . 31

A Lattice notation and conventions **32**

B Basic syntax for `MDP_QCD` **34**

C `Toy_class` and how it works **37**

D Safety rules **43**

1 Introduction

This paper describes `MDP_QCD`, a collection of classes and functions written in GNU[2] `C++` [1][2] to provide a framework to develop fast and efficient code for lattice simulation of $SU(N_c)$ gauge theories (with Wilson [3] or Sheikoleslami-Wolhert [4][5] fermions). For references see [6][7][8]

The main characteristics of `MDP_QCD` are:

- Implementation of the standard mathematical language used by physicists.
- It is based on `Toy_class` and `Matrix` (a class for matricial manipulations), both written by the author. The assignment operator and the copy constructor have been overloaded in a new way to reduce the necessity of copying when an object containing a pointer to dynamically allocated memory is returned by a function.
- Four main classes: `gauge_field`, `pl_field`, `em_field` and `fermi_field`, all based on the same optimization trick used in `Toy_class`. The basic binary operators have been overloaded for these classes.
- An advanced random number generator, including a generator of random $SU(N_c)$ matrices.
- Standard simulation algorithms: a quenched `multihit()` Metropolis montecarlo and a `mul_invQ()` to invert the fermionic matrix.
- Implementation of stochastic all-to-all propagators for light quarks.
- An optimized compressor for saving the gauge configurations.
- Everything works for an arbitrary number of colors, N_c.

The main idea on which the code has been built is the solution of the problem of returning objects containing pointers to dynamically allocated memory (realized in `class Matrix`, `Toy_class` and inherited by `gauge_field`, `pl_field`, `em_field` and `fermi_field`).

Consider the following code:

```
Matrix A,B(10,10);
A=B;
A=B*A;
```

[2] It has been tested with GNU `gpp` compiler on Solaris, Linux and Windows NT. Modifications may be needed for a different compiler.

In the first assignment one wants each element of `B` to be copied in the corresponding element of `A`. In the second assignment it is faster if `B` and `A` are passed to the local variables of the `operator*` by reference (i.e. without copying all the elements). Moreover one wants the local variable created by the `operator*` to occupy the same memory location as the variable that is returned (this avoids copying and wasting space). To implement this idea each `Matrix` object contains a `FLAG` and a pointer to dynamically allocated memory (where the numbers are stored). The copy constructor and the `operator=` have been overloaded in such a way to take care of the status of the `FLAG` and eventually to copy the pointer to the memory, instead of copying the memory containing the real matrix.

A physical location of memory may be pointed by different `Matrix` objects, but this never generates confusion if the safety rules (stated in appendix D) are followed. An automatic system of garbage collecting deallocates the unused memory when there are no objects alive pointing to it.

In this way, in the first assignment of the example, $11 + 800$ bytes[3] are copied, while in the second assignment only 11 bytes are copied three times (when passing `A` and `B` to the `operator*()` and when returning the result) to be compared with the 800 bytes of the matrix. The pointer of `A` is swapped only when the multiplication is terminated without generating confusion between the input and the output of the function. This is faster than it would be possible in FORTRAN. In FORTRAN it would be necessary to create a temporary array where to store the result of the multiplication and then copying it into `A`.

The `FLAG` takes care of everything and the procedure works in every possible recursive expression.

One more optimization is contained in the following code:

```
gauge_field U;
fermi_field psi;
psi(x)=U(x,mu)*psi(x);
```

In this example `U(x,mu)` and `psi(x)` are objects of `class Matrix` but each of them, instead of containing a pointer to new dynamically allocated memory, contains pointer to the same location of memory allocated by the constructor of `U` or `psi`. In other words: every time the programmer introduces a new object, the compiler knows if it is necessary to allocate new memory for it or just superimpose it to an existing allocated portion of memory.

All the details about these tricks are hidden from the high level programmer, who does not even need to know what pointers are.

[3]The number 11 is the size in bytes of a `Matrix` object. In is independent from the size of the memory occupied by the "real" matrix.

2 MDP_Lib1.h and its usage

Five classes are declared in the file MDP_Lib1.h [4]:

- Complex. Declared as complex<float>. The imaginary unit is implemented as a global constant I=Complex(0,1).
- Toy_class. It has no explicit application. It contains all the tricks to minimize the memory usage and maximize speed. Its members are inherited by the fields defined in MDP_QCD.h. The programmer does not need to know about the structure of Toy_class and for this reason its description is postponed to appendix C and D.
- Matrix. An object belonging to this class is effectively a complex matrix and it may have arbitrary dimensions. It is based on the same optimization techinique described in appendix C and D.
- Random_generator. This class contains public member functions to generate random long, float, double numbers and random $SU(n)$ matrices. Random is a global object belonging to this class and it can be used to gain access to any member function.
- JackBoot. It enables to compute Jackkinfe and Boostrap errors in a particularly easy and general Objcet Oriented way.

2.1 class Matrix

A matrix object, say M, can be declared in two different ways

```
Matrix M(r,c);     // r rows times c columns
Matrix M;          // a general matrix
```

Even if the size of a matrix has been declared it can be changed anywhere with the command

```
M.dimension(r,c);  // r rows times c columns
```

Any matrix is automatically resized, if necessary, when a value is assigned to it. The following lines are correct and print 8,8

```
Matrix M(5,7), A(8,8);
M=A;
printf("%i,%i\n", M.rowmax(),M.colmax());
```

[4] The classes defined in the file MDP_Lib1.C have already been used in some analysis programs written by the Southampton Theory Group [9][10][11][12].

The memeber functions `rowmax()` and `colmax()` return respectively the number of rows and columns of a `Matrix`.

The element (i, j) of a matrix `M` can be accessed with the natural syntax

```
M(i,j)
```

Moreover the class contains functions to perform standard operations:

```
+, -, *, /, +=, -=, *=, /=, inv, det, exp, sin, cos, log,
transpose, hermitiam, minor, identity,...
```

As an example a program to compute

$$\left[\exp\begin{pmatrix} 2 & 3 \\ 4 & 5i \end{pmatrix}\right]^{-1} \tag{1}$$

is the following

```
// Program saved in FILE: p01.C
#include "MDP_Lib1.h"
int main() {
   Matrix a(2,2);
   a(0,0)=2; a(0,1)=3;
   a(1,0)=4; a(1,1)=5*I;
   print(inv(exp(a)));
   return 0;
};
```

It is straightforward to add new functions copying the prototype [5]

```
Matrix f(Matrix a, Matrix b, ...) {
   Matrix M;
   // body,
   prepare(M);
   return M;
};
```

One more example of how to use the class `Matrix` follows

```
// Program saved in FILE: p02.C
#include "MDP_Lib1.h"
Matrix cube(Matrix X) {
   Matrix Y;
   Y=X*X*X;
   prepare(Y);
   return Y;
};
```

[5] A list of general safety rules to be applied when using the class `Matrix` can be found in Appendix D.

Example	C++ with MDP_Lib1.h
$A \in M_{r\times c}(\mathbf{C})$	`A.dimension(r,c)`
A_{ij}	`A(i,j)`
$A = B + C - D$	`A=B+C-D`
$A^{(ij)} = B^{(ik)}C^{(kj)}$	`A=B*C`
$A^{(ij)} = B^{(jk)}C^{(ik)}$	`A=mul_left(B,C)`
$A = aB + C$	`A=a*B+C`
$A = a\mathbf{1} + B - b\mathbf{1}$	`A=a+B-b`
$A = B^T C^{-1}$	`A=transpose(B)*inv(C)`
$A = B^\dagger \exp(iC)$	`A=hermitian(B)*exp(I*C)`
$A = \cos(B) + i\sin(B) * C$	`A=cos(B)+I*sin(B)*C`
$a = \mathrm{real}(tr(B^{-1}C))$	`a=real(trace(inv(B)*C))`
$a = \det(B) * \det(B^{-1})$	`a=det(B)*det(inv(B))`

Table 1: Examples of typical instructions acting on `Matrix` objects. `A,B,C,D` are assumed to be declared as `Matrix`; `r,c` as `int`; `a,b` may be any kind of number.

```
int main() {
   Matrix A,B;
   A=Random.SU(3);
   B=cube(A)*exp(A)+inv(A);
   print(A);
   print(B);
   return 0;
};
```

This code prints on the screen a random SU(3) matrix A and $B = A^3 e^A + A$. Some example statements are listed in tab. 1. Note the command

```
A=mul_left(B,C);
```

that is equivalent but faster than

```
A=C*transpose(B);
```

It will be useful to multiply a fermionic field (seen as a set of *color* $\times$ *spin* matries) by a spin structure.

2.2 `class Random_generator`

It is possible to declare different objects belonging to this class but this should be done only if one needs many independent random generators (in the sense that they relay on different seeds). For all the applications considered from now on,

only one random generator will be needed. To this scope a global object called `Random` is automatically declared when `MDP_Lib1.h` is included. Through this global object it is possible to access any member function of the class.

- `void Random.read_seed(char filename[]);` It reads the seed from the file `filename` and initializes the random generator. If no filename is provided the default filename is `RandomBuffer.seed`. If the program does not find the file with the seed it automatically creates it with some default values in it.

- `void Random.write_seed(char filename[]);` It writes the last computed seed into the file `filename`. The default file name is `RandomBuffer.seed`.

- `long Random.Long(long n);` It returns a random `long` number between `0` and `n-1`.

- `float Random.Float();` It returns a random `float` number between `0` and `1`.

- `double Random.Double();` It returns a random `double` number between `0` and `1`.

- `float Random.Gauss();` It returns a random `float` number x generated with a Gaussian probability $P(x) = \exp(-x^2/2)$.

- `float Random.Floatp(float (*P)(float, void*), void* a);` It returns a random number x generated with a Gaussian probability `P(x,a)`, where `a` is any set of parameters pointed by `a` (the second argument passed to `Floatp`).

- `Matrix Random.SU(int n);` it returns a random `Matrix` in the group $SU(n)$ or $U(1)$ if the argument is $n = 1$.

The algorithm for SU(2) is based on the observation that each rotation in the 3D space (in the group $O(3)$) can be represented by a direction $\widehat{a} \in S^2$ and a rotation angle around that direction, $\alpha \in [0, \pi)$. Therefore first a random element of $O(3)$ is generated, second, it is mapped to $SU(2)$. The map between $O(3)$ and $SU(2)$ is realized by

$$\{\widehat{a}, \alpha\} \rightarrow \exp(i\alpha\widehat{a} \cdot \sigma) = \cos(\alpha) + i\widehat{a} \cdot \sigma \sin(\alpha) \tag{2}$$

where $(\sigma^1, \sigma^2, \sigma^3)$ is a vector of Pauli matrices.

The algorithm for $SU(n > 2)$ is the Cabibbo-Marinari [13] iteration based on $SU(2)$.

2.3 class JackBoot

Suppose one has n gauge configurations and, on each configuration one has measured m float quantities a[0], a[1],...,a[m-1]. Then one wants to compute the average, over the gauge configurations, of a function F(float *a) with its Jackknife and Bootstrap errors [14]. A simple way to do it is to define a JackBoot object, let's call it jb. It is a sort of container for the data. After it has been filled it can be asked to return the mean of F() and its errors. Here is an example of how it works:

```
// object declaration:
JackBoot<float, m, n> jb;
// assigning the function:
jb.f=F; // note that f is a member variable
        // while F is the user defined function
// filling the object:
for(jb.elem=0; jb.elem<n; jb.elem++) {
   // somehow measure a[0],...a[m-1] on the gauge configs
   jb(0)=a[0];
   jb(1)=a[1];
   ...
   jb(m-1)=a[m-1];
};
// printing the results:
printf("Result          = %f\n", jb.mean());
printf("Jackknife error = %f\n", jb.j_err());
printf("Bootstrap error = %f\n", jb.b_err(100));
```

Note that

- The class JackBoot<...> jb is defined with a template where the first argument between angular brakets is the basic variable type (in the example it is float), the second is the size of the array to be passed to f(), the third is the number of confugurations to be processed.
- jb.f is the pointer to the function used in the analysis.
- jb.elem is an integer that must be used as a counter on the configurations.
- jb.operator(int n) is used to access data in the object (a configuration number in jb.elem must be specified).
- jb.mean() returns the mean.
- jb.j_err() returns the Jackknife error.
- jb.b_err() returns the Bootstrap error. It takes as argument the number of Bootstrap samples. The default value is 200.

It is important to stress that the class `JackBoot` can contain objects of any class `T`, but the functions `b_err` is not properly defined unless the operator `T::operator>(class T)` is declared. In other words one cannot perform a Boostrap of functions of matrices unless one defines some kind of ordering relation between matrices. I suggest to play safe and perform only Boostraps of `float` numbers: `JackBoot<float,n,m>`

It is possible to declare arrays of `JackBoot` objects, but it is rarely necessary. It is simpler to declare different functions and repeat the analysis using the same `JackBoot` object assigning the pointer `JackBoot::f` to each of the functions at the time.
As another example consider the following program. It generates an array of 100 $SU(6)$ matrices. For each matrix it computes trace and determinant, and returns the average of the ratio between the real part of the trace and the real part of the determinant (with its Jackknike and Bootstrap errors).

```
// Program saved in FILE: p03.C
#include "MDP_Lib1.h"
#define ng 100
float f(float *x) { return x[0]/x[1]; };
int main() {
   Matrix A;
   JackBoot<float,2,ng> jb;
   for(jb.elem=0; jb.elem<ng; jb.elem++) {
      A=Random.SU(6);
      jb(0)=real(trace(A));
      jb(1)=real(det(A));
   };
   jb.f=f;
   printf("Result          = %f\n", jb.mean());
   printf("Jackknife error = %f\n", jb.j_err());
   printf("Bootstrap error = %f\n", jb.b_err(100));
   return 0;
};
```

The output is

```
Result          = 0.036311
Jackknife error = 0.087398
Bootstrap error = 0.085979
```

3 MDP_QCD.h and its usage

3.1 Basic Syntax

The most general C++ program using MDP_QCD must have the structure

```
#include "MDP_QCD.h"

// declaration of functions

int main() {
   start();

   // initialize beta=...
   //            kappa=...
   //            c_SW=...
   // main body

   stop();
   return 0;
};
```

The file `MDP_QCD.h` includes `MDP_Lib1.h` and `MDP_Settings.h`. Any other header needed is included by `MDP_Lib1.h`.

The file `MDP_Settings.h` contains the declaration of those global constants which can be modified by the user: size of the lattice and the number of colors. A typical settings file is

```
#define Nx0    6  // temporal sites
#define Nx1    4  // spatial sites in the x direction
#define Nx2    4  // spatial sites in the y direction
#define Nx3    4  // spatial sites in the z direction
#define Nc     3  // number of colors
#define Nfermi 10 // used by light_propagator
```

Note that it is sufficient to change the value of `Nc` and recompile the code to change the gauge group. No other line has to be modified.

The instructions `start();` and `stop();` are compulsory and do the job of reading/writing the seed and initializating all the global variables and matrices. Remember that if the file containing the seed does not exist it is automatically created.

All the global matrices are listed in Appendix A together with the conventions used by `MDP_QCD`. Any other built-in global variable is listed in (57).

The basic objects in which lattice QCD is usually formulated are

- Matrices (`class Matrix`)
- Links (`class gauge_field`)
- Plaquettes (`class pl_field`)
- Chromo-electro-magnetic tensor (`class em_field`)
- Fermionic field (`class fermi_field`)

- Fermionic light propagator (`class light_propagator`)

To explain the power of this language, here is an example of a program to create one gauge configuration and test if the links are in $SU(N_c)$ (where N_c is declared in `MDP_Settings.h`)

```
// Program saved in FILE: p04.C
#include "MDP_QCD.h"
int main() {
   start();
   int mu, counter=0;
   site x;
   Matrix A;
   gauge_field U;
   U=hot();
   beta=5.7;
   multihit(U);
   for(x=0; x<Nvol; x++)
      for(mu=0; mu<4; mu++) {
         A=U(x,mu)*hermitian(U(x,mu))-1;
         if(real(trace(A*hermitian(A)))>0.0001)
             counter++;
      };
   if (counter==0) printf("All links are in SU(N)\n");
   stop();
   return 0;
};
```

It produces the following output:

```
============================================================
PROGRAM MDP_QCD. GENERAL INFORMATIONS:
============================================================
Created by Massimo Di Pierro (@ 1998)
Version n.     = 8-9-1998
Compiling date = Oct 16 1998
Compiling time = 20:57:58
Num. of colors = 3
Lattice size   = 6(t) x 4(x) x 4(y) x 4(z) = 384
============================================================
PROGRAM STARTING:
============================================================
Setting indices
Defining the matrices
Creating SU(N) generators
Readind Random.seed
Creating initial hot configuration...
Multihit step n.0, beta=5.700000...
... Efficiency: 83.40%
All links are in SU(N)
Writing Random.seed
```

```
=========================================================
PROGRAM END.
=========================================================
```

In the rest of this manual the follwing declarations will be assumed

```
int              i,j,k; \\ color indices
int              a,b,c; \\ spinor indices
int              mu,nu; \\ lattice directions
site             x,y,z; \\ lattice sites
Matrix           A,B,C; \\ generic matrices
gauge_field      U;
pl_field         P;
em_field         G;
fermi_field      chi, psi, phi;
light_propagator S;
```

The basic unary and binary operators have been overloaded. The use and power of the basic classes can be better explained with a the few example lines of tab. (2).

To handle properly objects belonging to any of the classes `gauge_field`, `pl_field`, `em_field` and `fermi_field` the safety rules described in Appendix D must be followed.

3.2 Creating gauge configurations

Any standard lattice simulation begins with the creation of an ensemble of gauge configurations $\{U^i\}$. It is created through a Markov process[7], i.e. each configuration U^i is generated from the preceding one, U^{i-1}, using any stochastic algorithm

$$U^i = F(U^{i-1}) \tag{3}$$

satisfying the relation

$$e^{\mathcal{S}[U^{i-1}]} P(U^{i-1} \to U^i) = e^{-\mathcal{S}[U^i]} P(U^i \to U^{i-1}) \tag{4}$$

where $P(U \to U')$ is the probability of generating the configurations U' from the configuration U. Here $\mathcal{S}[U]$ is the lattice Euclidean action evaluated on the gauge configuration U. In `MDP_QCD` the standard quenched action is implemented

$$\mathcal{S}[U] = \sum_x \frac{\beta}{N_c} \sum_{\mu,\nu} \mathrm{Re}\left[\ \mathrm{tr}\left(1 - U_\mu(x) U_\nu(x+\mu) U_\mu^\dagger(x+\nu) U_\nu^\dagger(x)\right)\right] \tag{5}$$

where $\beta = 1/g^2(a)$ is the parameter that fixes the lattice scale. The action in eq. (5) is a discretized version of the pure Yang-Mills action, i.e. fermion loops are neglected.

Example	C++ with MDP_QCD	returned type
$y = x + \mu$	`y=up[x][mu]`	`site`
$y = x - \mu$	`y=dw[x][nu]`	`site`
$y = x + 10\widehat{\mu}$	`y=move(x,mu,10)`	`site`
$y = x + 3\widehat{\mu} + 5\widehat{\nu}$	`y=move(x,mu,3,nu,5)`	`site`
$a = \|x - y\|$	`a=dist(x,y)`	`float`
$x = (x_0, x_1, x_2, x_3)$	`x=position(x0,x1,x2,x3)`	`site`
$U_\mu(x)$	`U(x,mu)`	`Matrix`
$U_\mu^{ij}(x)$	`U(x,mu,i,j)`	`Complex&`
$\psi(x)$	`psi(x)`	`Matrix`
$\psi_\alpha^i(x)$	`psi(x,i,alpha)`	`Complex&`
$U_\mu^{ij}(x) = U_\nu^{ik}(y)U_\rho^{kj}(z)$	`U(x,mu)=U(y,nu)*U(z,ro)`	`void`
$U_\nu(x)U_\mu(x+\widehat{\nu})U_\nu^H(x+\widehat{\mu})$	`staple_up(U,mu,nu)`	`Matrix`
$U_{-\nu}(x)U_\mu(x-\widehat{\nu})U_\nu(x+\widehat{\mu}-\widehat{\nu})$	`staple_dw(U,mu,nu)`	`Matrix`
$U_{\mu\nu}^\Box(x)$	`plaquette(U,x,mu,nu)`	`Matrix`
$\phi(x) = U_\mu(y)\psi(z)$	`phi(x)=U(y,mu)*psi(z)`	`void`
$(1+\gamma^\mu)\psi(z)$	`mul_left(1+Gamma[mu],psi(z))`	`Matrix`
$\sum_{x,i,\alpha}\psi_\alpha^i(x)\phi_\alpha^{i*}(x)$	`psi*phi`	`Complex`
$\psi = Q[U]\chi$	`psi=mul_Q(U,G,chi)`	`void`
$\psi = Q^{-1}[U]\chi$	`psi=mul_invQ(U,G,chi)`	`void`
$\psi = Q^\dagger[U]\chi$	`psi=mul_Q(U,G,chi,,DAGGER)`	`void`
$\psi = (Q^\dagger[U])^{-1}\chi$	`psi=mul_invQ(U,G,chi,,DAGGER)`	`void`

Table 2: Example lines in C++ with MDP_QCD. Note that $Q[...]$ is the fermionic matrix derived from the Sheikoleslami-Wolhert action. Its explicit expression is given in eq. (18).

The initial configuration U^0 can be chosen to be "cold", i.e. when all its links are the identity, or "hot", when each link is a random $SU(N)$ matrix.

Since it is very important to reduce the correlation between U^{i-1} and U^i, it is necessary to iterate eq.(3) many times

$$U^i = F(F(...F(U^{i-1})...)) \tag{6}$$

Any quantum gauge observable $O[U]$ can be measured as an an average over the ensemble of gauge configurations

$$\langle O[U] \rangle = \int [dU] O[U] P[U] \simeq \frac{1}{N_U} \sum_i O[U^i] \tag{7}$$

In `MDP_QCD` a gauge configuration U is an object of the class `gauge_field`. It is declared with the command

```
gauge_field U;
```

An initial hot configuration is created with the command

```
U=hot();
```

and an initial cold configuration is created with the command

```
U=cold();
```

The operation described in eq.(3) and eq.(6) is performed using the multihit algorithm that is described in the section on algorithms. The command to perfrom it is

```
// beta=...
multihit(U,num_hits,num_iter);
```

`num_hits` is the number of hits for each multihit step. `num_iter` is the number of multihit iterations. A value for the gloabal variable `beta` must be specified. The new configuration is stored in the same object U which contains the configuration which is passed as argument. This saves memory. In principle it is possible to define arrays of configurations and define many different gauge configurations at the same time

```
gauge_field myconf[100][3];
gauge_field U, U1, U2, U3;
```

and copy one into the other as if they were ordinary variables

```
myconf[37][2]=U;
U2=U;
U=myconf[50][0];
```

To avoid ambiguities the assignment operator of any kind of field returns void.

Links $U_\mu(x)$ can be accessed as matrices `U(x,mu)` or as complex numbers $U_\mu^{ij}(x)$, `U(x,mu,i,j)`. It is also possible to access a link as a matrix and extract its (i,j) element `U(x,mu)(i,j)` but this is slower than the direct access `U(x,mu,i,j)`.

Here is a program that generates and saves an ensemble of 100 gauge configurations, at $\beta = 6$, starting from hot, performing 20 multi-hit steps at the time

```
// Program saved in FILE: p05.C
#include "MDP_QCD.h"
int main() {
   start();
   int g;
   gauge_field U;
   U=hot();
   beta=6.0;  // beta is declared in MDP_QCD.h
   for(g=0; g<100; g++) {
      multihit(U,5,20);
      write(U, "gauge_config_beta_6.0", g);
   };
   stop();
   return 0;
 };
```

3.3 First measurements

It only makes sense to measure gauge invariant quantities, i.e. closed loops of links. The minimum closed loop is the plaquette (`pl`)

$$P_{\mu\nu}(x) = U_\mu(x)U_\nu(x+\widehat{\mu})U_{-\mu}(x+\widehat{\mu}+\widehat{\nu})U_{-\nu}(x+\widehat{\nu}) \tag{8}$$

which is a rank two tensor field. It is related to the chromo-electro-magnetic (`em`) tensor field through the relation

$$G_{\mu\nu}(x) = \frac{1}{8}\left[P_{\mu\nu}(x) + P_{\mu\nu}(x-\widehat{\mu}) + P_{\mu\nu}(x-\widehat{\nu}) + P_{\mu\nu}(x-\widehat{\mu}-\widehat{\nu})\right] \tag{9}$$

$$- \frac{1}{8}\left[P_{\nu\mu}(x) + P_{\nu\mu}(x-\widehat{\mu}) + P_{\nu\mu}(x-\widehat{\nu}) + P_{\nu\mu}(x-\widehat{\mu}-\widehat{\nu})\right] \tag{10}$$

In `MDP_QCD` there is a class for the plaquettes field

```
pl_field P;
```

and a class for the chromo-electro-magnetic field

```
em_field G;
```

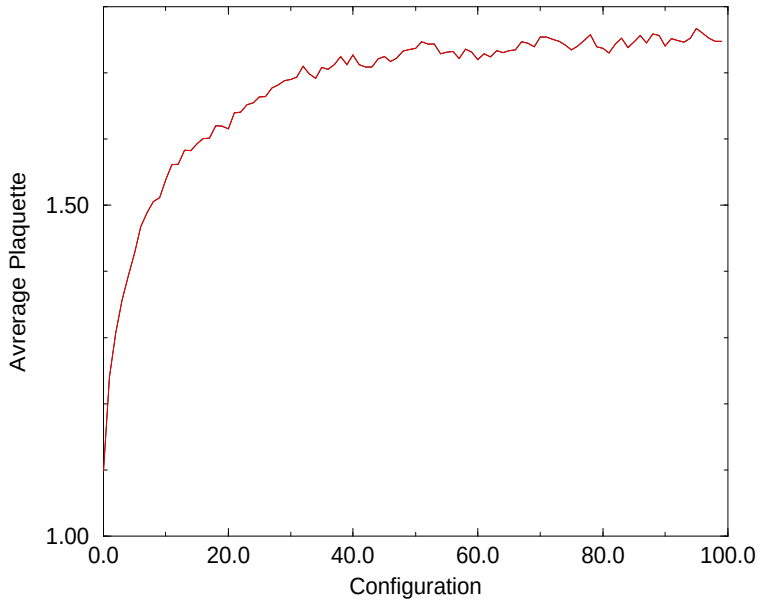


Figure 1: The graph shows the real part of the trace of the average plaquette computed on 100 gauge configurations. The plateau indicates the termalized configurations.

For an arbitrary gauge configuration U these field are computed with the commands

```
P=compute_plaquettes(U);
G=compute_em_tensor(P);
```

An interesting measurement to be performed on each gauge configuration is the real part of the trace of the average plaquette:

```
float av_pl;
av_pl=average(P);
```

What follows is a program that reads the gauge configurations created by `p05.C`, computes the average plaquette and prints the result for each gauge configuration.

```
// Program saved in FILE: p06.C
#include "MDP_QCD.h"
int main() {
  start();
  int g;
  gauge_field U;
  for(g=0; g<100; g++) {
    read(U, "/home/lattice/mdp/gauge_config_beta_6.0", g);
    printf("%i %f\n",g, average(compute_plaquettes(U)));
  };
```

```
  stop();
  return 0;
};
```

The output of the program has been plotted and is shown in fig. (1).

It shows that the average plaquette approaches an asymptotic value (which mainly depends on the value of β) and then fluctuates around it. This is the region where the field is said to be thermalized, i.e. when the statistical fluctuations compensate the classical tendency to minimise the free energy. This is the region that corresponds to a physical quantum vacuum. Configurations that are not thermalized should be discarded. As is evident from the figure, in our case the first 60 saved configurations are not termalized. We will refer to the remaining 40 as "effective configurations".

A `pl_field P` can be accessed as $N_c \times N_c$ matrix `P(x,mu,nu)` or as complex number `P(x,mu,nu,i,j)`. Analogously an `em_field G` can be accessed as an $N_c \times N_c$ matrix `G(x,mu,nu)` or as complex number `G(x,mu,nu,i,j)`. **In both cases `mu` must be less than `nu`. The other components can be obtained by antisymmetrizing.**

For a general review on the typical quantities that can be computed on lattice see ref. [15].

As an example the next program computes a Polyakov loop on our effective 40 gauge configurations and prints out the result for each gauge configuration.

```
// Program saved in FILE: p07.C
#include "MDP_QCD.h"
int main() {
  start();
  int g,x0;
  site x;
  gauge_field U;
  Matrix Polyakov(Nc,Nc);
  for(g=60; g<100; g++) {
    read(U, "gauge_config_beta_6.0", g);
    Polyakov=1;
    x=0;
    for(x0=0; x0<Nx0; x0++) {
      Polyakov=Polyakov*U(x,0);
      x=up[x][0];
    };
    print(Polyakov);
  };
  stop();
  return 0;
};
```

3.4 Introducing fermions

Another set of observables that can be measured on the lattice are the two and three point correlation functions between currents (and their Fourier transform)

$$C_2(t_x) = \int d^3\mathbf{x}\langle 0|J(0)J^\dagger(x)|0\rangle \tag{11}$$

$$C_3^Q(t_x,t_y) = \int d^3\mathbf{x}d^3\mathbf{y}\langle 0|J(-y)Q(0)J^\dagger(x)|0\rangle \tag{12}$$

Since the lattice metric is Euclidean, the asymptotic behaviour of the spatial Fourier transform of the two point correlation function is given by

$$C_2(t_x) \sim Z^2 e^{-m_J t_x} \tag{13}$$

where m_J is the mass of the lightest state $|1_J\rangle$ created by the current $J^\dagger$ and

$$Z = |\langle 1_J|\, J^\dagger(0)\,|0\rangle| \tag{14}$$

Therefore from the measurement of $C_2(t_x)$ and its fit to (13), it is possible to extract masses of particles. In the same fashion from the asymptotic behaviour of the ratio between the three and two two point correlation functions it is possible to extract matrix elements[16]

$$\frac{C_3^Q(t_x,t_y)}{C_2(t_x)C_2(t_y)} \sim \frac{1}{Z^2}\frac{\langle 1_J|Q|1_J\rangle}{2m_J} \tag{15}$$

The most general current $J(x)$ can be expressed in terms of fundamental fermionic fields $q_\alpha^i(x)$ (the quark fields). In expressions like eq.(11) and (12) these field are Wick contracted

$$\langle 0|\{q_\alpha^i(x),\overline{q}_\beta^j(y)\}|0\rangle = S_{\alpha\beta}^{ij}(x,y) \tag{16}$$

Despite the fact that fermions are neglected when gauge configurations are created, they are reintroduced at a later stage as particles propagating in the gluonic background field. Therefore the two and three point correlation functions can be written as appropriate traces of propagators, $S_{\alpha\beta}^{ij}(x,y)[U]$, in the backgroud gluonic field U.

In `MDP_QCD` fermionic fields belong to the class `fermi_field`. In the following line the fields χ, ψ and ϕ are declared

```
fermi_field chi, psi, phi;
```

On each gauge configuration U, the fermion propagator S is computed by inverting the fermionic matrix

$$S[U] = (Q[U])^{-1} \tag{17}$$

This is the most time expensive part of any lattice calculations. The fermionic matrix implemented comes from the standard Sheikoleslami-Wolhert action[6] [22]

$$\begin{aligned} Q_{x,y}[U] &= \delta_{x,y} - \kappa \sum_{\mu=0..3} \left[(1-\gamma_\mu)U_\mu(x)\delta_{x,y-\mu} + (1+\gamma_\mu)U^\dagger_\mu(x-\mu)\delta_{x,y+\mu}\right] \\ &- i\kappa c_{SW} \sum_{\mu<\nu} \sigma_{\mu\nu} G_{\mu\nu}(x)\delta_{x,y} \end{aligned} \tag{18}$$

where κ and c_{SW} are input parameters. κ is in one to one correspondence with the fermion mass

$$m = \frac{1}{2a}\left(\frac{1}{\kappa} - \frac{1}{\kappa_{crit}}\right) \tag{19}$$

and κ_{crit} is a parameter depending on β. The chiral limit corresponds to the limit $\kappa \to \kappa_{crit}$, when the quark becames massless. In practice any inversion algorithm for eq.(17) converges slower and slower as the chiral limit is approached and this can never be reached. c_{SW} also depends on β and its purpose is to cancel order a effects in the propagator (as well as in physical observables). c_{SW} can be computed perturbatively (it is 1 at three level), can be extimated from the average plaquette (equivalent to resumming "tadpole" contributions) or can be obtained by a fine tuning procedure (called "non-perturbative" improvement).

κ and c_{SW} are declared as global variables in `MDP_QCD`. The multiplication of a `fermi_field` by the fermionic matrix is performed by the function `mul_Q()` [7].

```
kappa=0.135;
c_SW=0;
G=compute_em_tensor(compute_plaquettes(U));
psi=mul_Q(U, G, chi);
```

Here the second line reads

$$\psi = Q[U]\chi \tag{20}$$

Note that the chromo-electro-magnetic field $G_{\mu\nu}$ computed on the gauge configuration U must be passed to the function `mul_Q`. This is done to speed up the computation of the so called "clover" term (the part of the action proportional to c_{SW}).

The inverse operation is performed by the function

```
chi=mul_invQ(U, G, psi, 1e-4);
```

[6] The Feynman rules derived for this action can be found in [17]. Note that in this paper the $\sigma_{\mu\nu}$ matrices are different, for an i factor, from those implemented and listed in appendix A.

[7] Note that before calling `mul_Q()` it is necessary to assign a value to the global variables `kappa` and `c_SW`

The fourth argument is the required precision in the inversion. By default it is 10^{-4}.

Consider now the following program

```
// Program saved in FILE: p08.C
#include "MDP_QCD.h"
int main() {
  start();
  float precision=1e-4;
  gauge_field U;
  em_field G;
  fermi_field chi, psi, phi;
  U=hot();
  beta=6.0;
  multihit(U,1,20);
  G=compute_em_tensor(compute_plaquettes(U));

  kappa=0.125;
  c_SW=1.77;
  chi=new_fermi_config(U,G);

  psi=mul_Q(U,G,chi);
  phi=mul_invQ(U,G,psi,precision);
  phi=phi-chi;
  precision=abs(phi*phi)/(Nvol*Nc*Nc);
  printf("precision=%f\n",precision);
  stop();
  return 0;
};
```

It creates a gauge configuration U, a random fermionic configuration χ and computes

$$\psi = Q[U]\chi \tag{21}$$

$$\phi = Q^{-1}[U]\psi - \chi \tag{22}$$

$$\delta = \sum_{i,\alpha} \int d^4x \left[\phi^i_\alpha(x)\right]^* \phi^i_\alpha(x) \tag{23}$$

It then checks whether δ is consistent with the input precision. Note the command

```
chi=new_fermi_config(U,G);
```

it returns a random fermi_field χ with probability

$$P[\chi] = \exp\left[-\chi^\dagger Q^\dagger[U]Q[U]\chi\right] \tag{24}$$

Again, a `fermi_field psi` can be accessed as an $N_c \times 4$ matrix, `psi(x)` or as complex number `psi(x,i, alpha)`.

A more difficult task is the computation of the propagator of eq. (16) from all points to all points.

3.5 Stochastic light propagators

The light propagator of eq. (16) can be computed, for each gauge configuration, as an average [18]

$$S^{ij}_{\alpha\beta}(x,y)[U] = \left\langle \varphi^{[k]i}_{\alpha}(x) \left(\chi^{[k]j}_{\beta}(y)\right)^{*} \right\rangle_{k} \tag{25}$$

where for each k

$$\chi^{[k]} = Q[U]\varphi^{[k]} \tag{26}$$

The `fermi_field` configurations $\varphi^{[k]}$ must be generated stochastically [8] with probability

$$P[\varphi] = \exp\left(-\varphi^{\dagger}Q^{\dagger}[U]Q[U]\varphi\right) \tag{27}$$

In `MDP_QCD` light propagators are objects of the class `light_propagator`. A typical light propagators, S, is declared as

```
light_propagator S;
```

Each `light_propagator S` contains two member arrays of `fermi_field`

```
fermi_field S.psi[Nfermi];
fermi_field S.chi[Nfermi];
```

These arrays can be created[9] for a fixed gauge configuration, with the command:

```
S.stochastic_generation(U,G);
```

where `U` is the gauge field and `G` if the electromagnetic tensor. Here is how it is implemented

```
light_propagator::stochastic_generation(gauge_field U, em_field G) {
  register site k;
  printf("Start stochastic generation: Nfermi=%i\n", Nfermi);
    for(k=0; k<Nfermi; k++) {
    printf("Step %i:",k);
    psi[k]=new_fermi_config(U,G,0.0001);
    chi[k]=mul_Q(U,G,psi[k]);
  };
  printf("Stochastic generation terminated.\n");
};
```

[8] The technique for generating these configurations has been recently carried one step forward and combined with a "maximal variance reduction" method to reduce the statistical noise [19]. This method is not implementd in `MDP_QCD`.

[9] Note that before generating or reading a `light_propagator` it is necessary to assign a value to the global variables `kappa` and `c_SW` which are used by `mul_Q()`.

This code is the time expensive, therefore it may be a good idea to save the fermionic configurations once they have been generated.

In the following program a stochastic propagator is created and saved for the 60th of our gauge configurations.

```
// Program saved in FILE: p09.C
#include "MDP_QCD.h"
int main() {
   start();
   gauge_field U;
   em_field G;
   light_propagator S;
   read(U, "gauge_config_beta_6.0", 60);
   kappa=0.1345; c_SW=1.77;
   G=compute_em_tensor(compute_plaquettes(U));
   S.stochastic_generation(U,G);
   write(S,"propagator.1345.177.", 60);
   stop();
   return 0;
};
```

There are two basic ways to extract informations from a `stochastic_propagator` `S`.

- `S(x,i,y,j,k)` returns the spin Matrix $N_{spin} \times N_{spin}$

$$\varphi_\alpha^{[k]i}(x) \left(\chi_\beta^{[k]j}(y)\right)^* \tag{28}$$

- `S(x,i,y,j)` returns the same spin Matrix $N_{spin} \times N_{spin}$ averaged on the spinorial configurations.

$$\left\langle \varphi_\alpha^{[k]i}(x) \left(\chi_\beta^{[k]j}(y)\right)^* \right\rangle_k \tag{29}$$

The following program computes, for just one gauge configuration, the two point correlation function of eq. (11) when $J(x)$ is the current $\overline{b}\gamma^5 q$. Note that the propagator for the b is computed in the static approximation, i.e. as product of links times $\frac{1}{2}(1+\gamma^0)$. The light propagator is read from a file.

```
// Program saved in FILE: p10.C
#include "MDP_QCD.h"
int main() {
  start();
  gauge_field U;
  em_field    G;
  light_propagator S;
  site x,y,z;
  int i,j,L=2;
```

```
  Matrix H(Nc,Nc);
  Matrix A;
  Complex result=0;
  read(U, "gauge_config_beta_6.0", 60);
  G=compute_em_tensor(compute_plaquettes(U));
  kappa=0.1345; c_SW=1.77; // They must correspond to the S saved!
  read(S,U,G,"propagator.1345.177.",60);
  x=0; y=move(x,0,L);
  H=1;
  A=Gamma5*0.5*(1+Gamma[0])*Gamma5
  for(z=x; z!=y; z=up[z][0]) H=H*hermitian(U(z,0));
  for(i=0; i<Nc; i++)
    for(j=0; j<Nc; j++)
      result=result+trace(S(y,i,x,j)*A)*H(j,i);
  printf("result=%f + %fI\n", real(result), imag(result));
  stop();
  return 0;
};
```

Some care is necessary when simulating a process with more than one light quark. The safe way is to define many stochastic propagators corresponding to the different contractions. To save memory and time it is possible to use just one `light_propagator` to compute all of the "physical" propagators (assuming the corresponding quarks are degenerate in mass). In this case the average of eq. (25) cannot be computed independently for each of the propagators and unwanted correlations must be somehow eliminated.

3.6 Smearing

To improve the overlap between a current, $J^\dagger$, and the physical state one wants to study, $|1_J\rangle$, it is common to smear the light propagator. To this scope the gauge invariant "Wupperthal" smearing[20] is implemented

```
float epsilon=0.25;
int   num_iter=60;
psi=smearing(chi,epsilon,num_iter);
```

This smears the χ field with a smearing parameter ϵ, iterates the procedure `num_iter` times, then saves the smeared field in ψ.

Here is how the function `smearing` is implemented

```
fermi_field smearing(fermi_field psi, gauge_field U,
                     float epsilon, int level) {
  fermi_field phi;
  int n;
  register site x;
  printf("Smearing fermionic configuration...\n");
  for(n=0; n<level; n++) {
```

```
    if(n>0) psi=phi;
    for(x=0; x<Nvol; x++)
      phi(x)=psi(x)+epsilon*
        (U(x,1)*psi(up[x][1])+
         U(x,2)*psi(up[x][2])+
         U(x,3)*psi(up[x][3])+
         hermitian(U(dw[x][1],1))*psi(dw[x][1])+
         hermitian(U(dw[x][2],2))*psi(dw[x][2])+
         hermitian(U(dw[x][3],3))*psi(dw[x][3]));
  };
  prepare(phi);
  return phi;
};
```

To smear the sink (and/or the source) of a stochastic propagator it is necessary to apply the smearing function to its member fields

```
for(n=0; n<Nfermi; n++) {
   S.psi[n]=smearing(S.psi[n],epsilon,num_iter); // smear sink
   S.chi[n]=smearing(S.chi[n],epsilon,num_iter); // smear source
}
```

3.7 Examples: fuzzing and cooling

Fuzzing is a kind of gauge inveriant smearing that is not explicitely implemented in `MDP_QCD` but there are different ways of doing it. It essentially consists of performing an ordinar smearing using a a "fuzzed" gauge configuration instead of the normal one. Here is how it could be implemented

```
// EXAMPLE CODE: function not implemented!
fermi_field fuzzing(fermi_field psi, gauge_field U, int num_iter) {
  fermi_field phi;
  gauge_field Ufuzzed;
  site x;
  int mu,i;
  for(i=0; i<num_iter; i++)
    for(x=0; x<Nvol; x++)
      for(mu=0; mu<Ndim; mu++) {
        Ufuzzed(x,mu)=project_on_SUn(U(x,mu)+0.25*staple(U,x,mu));
    };
  phi=smearing(psi,Ufuzzed,0.25,2);
  prepare(phi);
  return phi;
};
```

The function `project_on_SUn` is not implemented in `MDP_QCD` and there are different ways to do it. Its job is to return some kind of projection of the matrix, which is passed as argument, on the $SU(N_c)$ group. This procedure is called "cooling". One way of implementing it is

```
// EXAMPLE CODE: function not implemented!
Matrix project_on_SUn(Matrix A) {
  Matrix logA, B(Nc,Nc);
  int i;
  B=0;
  logA=log(A);
  for(i=1; i<=Nadj; i++)
    B+=Generator[i]*real(0.5*trace(Generator[i]*logA));
  B=exp(B);
  prepare(B);
  return B;
};
```

Note that `Generator[i]` form a basis of $SU(N_c)$ and are built in in `MDP_QCD`. This code is very slow, but it can be speeded up by decreasing the precision in the computation of `log()` and `exp()` for matrices. This is set in the flag `PRECISION` declared in the file `MDP_Lib1.h`. In any case this is usually not the critical part of a lattice simulation.

3.8 Basic algorithms: `multihit()`

The multihit code consists of the following iteractive procedure [7][21]

- ▶ For each link of the lattice
- (1) Generate a random matrix
- (2) Compute the variation in the action, δS
- (3) If $\delta S > x$, where x is random real number, substitute the new link to the old one
- ↻ Repeat steps $\{1, 2, 3\}$ `n_hits` times
- ↻ Repeat from ▶ `n_iter` times

This algorithm is implemented in the following code

```
void multihit(gauge_field U, int n_hit=10, int n_iter=1) {
   register int     i,hit,mu,nu;
   register site    x;
   register double  delta_action, efficency;
   long    updated;
   Matrix  Unew,UUU;
   for(i=0; i<n_iter; i++) {
      printf("Multihit step n.%i, beta=%f...\n",i,beta);
      updated=0;
      for(x=0; x<Nvol; x++) {
         for(mu=0; mu<4; mu++) {
```

```
            UUU=staple(U,x,mu);
            for(hit=0; hit<n_hit; hit++) {
               Unew=Random.SU(Nc)*U(x,mu);
               delta_action=exp(beta/Nc*
                  real(trace(UUU*hermitian(Unew-U(x,mu)))));
             if(delta_action>Random.Float()) {
                  U(x,mu)=Unew;
                  updated++;
                  hit=n_hit;
               };
            };
         };
      };
      efficency=100.0*updated/(Nvol*Ndim);
      printf("... Efficency: %.2f%%\n", efficency);
   };
};
```

The parameter called `efficiency`, computed by this function and printed on the standard output, is the ratio between the number of links that has been updated on each multi-hit step and the total number of links.

3.9 Basic algorithms: `mul_Q()` and `mul_invQ()`

The function `mul_Q()` performs multiplication by the fermionic matrix as it appears in the SW action. Here is a possible implementation:

```
fermi_field mul_Q(gauge_field U, em_field G,
                  fermi_field psi, int sign=1) {
   // Slow Version!
   site x;
   int mu,nu;
   fermi_field tmp;
   tmp=psi;
   for(x=0; x<Nvol; x++)
      for(mu=0; mu<Ndim; mu++) {
         tmp(x)-=kappa*mul_left(1-sign*Gamma[mu],
                  U(x,mu)*psi(up[x][mu]))
       +kappa*mul_left(1+sign*Gamma[mu],
          hermitian(U(dw[x][mu],mu))*psi(dw[x][mu]));
         for(nu=mu+1; nu<Ndim; nu++)
    tmp(x)-=kappa*c_SW*left_mul(Sigma[mu][nu],
  G(x,mu,nu)*psi(x));
      };
   prepare(tmp);
   return tmp;
};
```

This would work but, in practice, the real implementation has been optimized to reduce the number of loops.

In the real implementation the multiplication when $c_{SW} \neq 0$ is only 80% slower than when $c_{SW} = 0$ (because the "clover" term is not computed).
The command

```
psi=mul_invQ(U, G, chi, delta);
```

returns $\psi = Q^{-1}[U]\chi$. The inversion is performed using the technique of minimal residue [8]. Note that the electromagnetic tensor `G` is passed as an argument as well. This is redundant but it is much faster if many inversions have to be performed on the same gauge configuration and c_{SW} is different from zero.

The minimal residue scheme consists of the following iterative procedure

- ▶ $\phi_0 = \chi$, $r_0 = \chi - Q[U]\chi$
- (1) $\alpha_i = \frac{(Q[U]r_i)\cdot r_i}{(Q[U]r_i)\cdot(Q[U]r_i)}$
- (2) $\phi_{i+1} = \phi_i + \alpha_i r_i$, $r_{i+1} = r_i - \alpha_i Q[U]r_i$
- ↻ iterate step $\{1, 2\}$ until $|r_i|^2$ is less then the required precision (`delta/(Nvol*Nc*Nc)`).
- ■ ϕ_i converges towards ψ.

Here is how it is implemented

```
fermi_field mul_invQ(gauge_field U, em_field G, fermi_field chi,
                     float delta_res=0.0001, int sign=1) {
  printf("Inverting the Q matrix with kappa=%f, c_SW=%f ...\n",
         kappa, c_SW);
  fermi_field psi, res, Qres;
  float residue;
  Complex alpha;
  int step=0;
  psi=chi;
  res=chi-mul_Q(U,G,chi,sign);
  do {
    Qres=mul_Q(U,G,res,sign);
    alpha=(Qres*res)/(Qres*Qres);
    psi=psi+alpha*res;
    res=res-alpha*Qres;
    residue=real(res*res)/(Nvol*Nc*Nc);
    step++;
    printf("   Step %i =>%f\n", step, residue);
  } while (residue>delta_res);
  printf("... %i steps, precision reached=%e\n", step, residue);
  prepare(psi);
  return psi;
};
```

3.10 Input/Output

One gauge configuration `U` can be saved with the command

```
write(U, "filename", g);
```

where `g` is an integer. Assuming for example that `g` is 36, the configuration is saved in a file called `filename.00036`. The command to load this file in a gauge configuration `U` is

```
read(U, "filename", g);
```

Using properties of $SU(N_c)$ the configuration is compressed when it is saved and automatically decompressed when it is read. After the compression each link occupies $12(N_c^2 - N_c)$ bytes instead of $16N_c^2$.

Fermionic configurations are saved and loaded with the commands

```
write(psi, "filename", g, n);
read(psi,  "filename", g, n);
```

to/from a file `filename.`$g.n$.

A `light_progator S` is saved with the command

```
write(S, "filename", g);
```

which calls

```
for(n=0; n<Nfermi; n++) write(S.psi[n], "filename", g, n);
```

It can be read with

```
G=compute_em_tensor(compute_plaquettes(U));
read(S, U, G, "filename",g);
```

Note that it is necessary to pass the gauge field and the electromagnetic field to this function. They are necessary to rebuild the `S.chi[Nfermi]` fields from the `S.psi[Nfermi]`. U must be exactly the same gauge configuration on which the `S.phi[Nfermi]` fields had been generated.

object class	memory (bytes)
`Matrix`	$11 + rows \times columns$
`gauge_field`	$5 + 32 \times N_{volume} \times N_c^2$
`pl_field`	$5 + 96 \times N_{volume} \times N_c^2$
`em_field`	$5 + 96 \times N_{volume} \times N_c^2$
`fermi_field`	$5 + 32 \times N_{volume} \times N_c$
`light_propagator`	$(5 + 32 \times N_{volume} \times N_c) \times 2 \times N_{fermi}$

Table 3: Memory usage for the different fields. N_{volume} is for the total number of lattice sites.

lattice	$U(1)$	$SU(2)$	$SU(3)$	$SU(4)$	$SU(5)$
$2^3 \times 3$	0.03	0.03	0.10	0.33	0.47
$4^3 \times 6$	0.28	0.50	1.21	2.72	5.99
$6^3 \times 9$	1.25	2.65	6.18	14.54	32.94
$8^3 \times 12$	3.84	8.02	20.07	45.18	93.65
$12^3 \times 18$	20.14	38.79	98.07	-	-

Table 4: Time in seconds to perfrom one multihit step (1 hit per link) on a SUN UltraSPARC 5.

lattice	$U(1)$	$SU(2)$	$SU(3)$	$SU(4)$	$SU(5)$
$2^3 \times 3$	0.05	0.07	0.13	0.24	0.43
$4^3 \times 6$	0.46	1.08	2.20	4.19	6.36
$6^3 \times 9$	2.24	5.34	11.20	19.37	31.95
$8^3 \times 12$	7.14	14.87	35.80	62.24	105.00
$12^3 \times 18$	35.74	84.32	174.90	-	-

Table 5: Time in seconds to generate one random fermionic configuration (it involves one call to `mul_invQ()` with a precision of 10^{-4}), far from the chiral limit. Computed on a SUN UltraSPARC 5.

3.11 Memory and speed

Tab. (3) shows the required memory to store each of the fundamental objects (as function of the lattice size defined in `MDP_settings.h`).

The function `mul_invQ()` allocates 3 `fermi_field` and one of them is returned.

All the other functions have been optimized and essentially they do not use more memory than that allocated to their arguments plus the memory required to store the argument that is returned. It is important to remeber that when a field is returned it is NOT copied, but only the pointer to the physical memory is copied. In other words the memory occupied by a field object that has to be returned by a function is the same memory where the field will be stored after the function has returned.

Since the time required to copy memory has been reduced to its minimum any function declared in `MDP_QCD.h` is very fast. `MDP_QCD` enables to run significative lattice simulations on a SUN UltraSPARC or on a Pentium II PC. Some benchmarks executed are reported in tab. (4) and tab. (5) for the most common operations.

It is important to stress that `MDP_QCD` does not include any optimization for parallel machines and many of the trick that are implemented may result in a slowing down on parallel computers without shared memory.
Internally all the fields and matrices are seen as one dimensional arrays of `Complex`. This gives some benefit on vectorial machines providing the compiler is able to optimize it.

A Lattice notation and conventions

- Equivalence between Lattice Euclidean and Minkowsky Space quantities (where a is the lattice spacing).

Euclidean Lattice	Minkowsky Space
x^0	$ia^{-1}x^0$
x^i	$a^{-1}x^i$
∂^4	$-ia\partial_0$
∂^i	$a\partial_i$
A^0	$-iaA_0$
A^i	aA_i
F^{0i}	$-ia^2F_{0i}$
F^{ij}	a^2F_{ij}
γ^0	γ^0
γ^i	$-i\gamma^i$
γ^5	γ^5

(30)

- Metric

$$g^{\mu\nu} = -\delta^{\mu\nu} = \mathrm{diag}(-1,-1,-1,-1) \tag{31}$$

- Pauli matrices

$$\texttt{sigma[1]} \;=\; \sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \tag{32}$$

$$\texttt{sigma[2]} \;=\; \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \tag{33}$$

$$\texttt{sigma[3]} \;=\; \sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{34}$$

they undergo the commutation relation

$$[\sigma_i, \sigma_j] = 2i\varepsilon^{ijk}\sigma_k \tag{35}$$

- Dirac matrices in Dirac representation (the same convention of [7] and UKQCD)

$$\texttt{Gamma[0]} \;=\; \gamma^0 = \begin{pmatrix} \mathbf{1} & 0 \\ 0 & -\mathbf{1} \end{pmatrix} \tag{36}$$

$$\texttt{Gamma[i]} \;=\; \gamma^i = \begin{pmatrix} 0 & -i\sigma_i \\ i\sigma_i & 0 \end{pmatrix} \tag{37}$$

$$\texttt{Gamma5} \;=\; \gamma^5 = \begin{pmatrix} 0 & \mathbf{1} \\ \mathbf{1} & 0 \end{pmatrix} \tag{38}$$

$$\texttt{Gamma1} = \delta^{\mu\nu} = \frac{1}{2}\{\gamma^\mu, \gamma^\nu\} \tag{39}$$

$$\texttt{Sigma[mu][nu]} = \sigma^{\mu\nu} = \frac{i}{2}[\gamma^\mu, \gamma^\nu] \tag{40}$$

$$\texttt{Pleft} = P_{left} = \frac{1-\gamma^5}{2} \tag{41}$$

$$\texttt{Pright} = P_{right} = \frac{1+\gamma^5}{2} \tag{42}$$

Note that all the γ and σ matrices are hermitian and, by definition, $\gamma^5 = \gamma^0\gamma^1\gamma^2\gamma^3$.

- Traces

$$tr(\gamma^\mu\gamma^\nu) = 4\delta^{\mu\nu} \tag{43}$$

$$tr(\gamma^\mu\gamma^\nu\gamma^\rho) = 0 \tag{44}$$

$$tr(\gamma^\mu\gamma^\nu\gamma^\rho\gamma^\sigma) = 4(\delta^{\mu\nu}\delta^{\rho\sigma} - \delta^{\mu\rho}\delta^{\nu\sigma} + \delta^{\mu\sigma}\delta^{\rho\nu}) \tag{45}$$

$$tr(\gamma^5\gamma^\mu\gamma^\nu\gamma^\rho\gamma^\sigma) = 4\epsilon^{\mu\nu\rho\sigma} \tag{46}$$

where $\epsilon^{0123} = \epsilon^{0123} = -1$.

- Gell-Mann matrices, $\texttt{Lambda[i]} = \lambda_i$

$$\lambda^1 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \qquad \lambda^2 = \begin{pmatrix} 0 & -i & 0 \\ i & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \tag{47}$$

$$\lambda^3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{pmatrix} \qquad \lambda^4 = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} \tag{48}$$

$$\lambda^5 = \begin{pmatrix} 0 & 0 & -i \\ 0 & 0 & 0 \\ i & 0 & 0 \end{pmatrix} \qquad \lambda^6 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \tag{49}$$

$$\lambda^7 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -i \\ 0 & i & 0 \end{pmatrix} \qquad \lambda^8 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -2 \end{pmatrix} \tag{50}$$

- Discrete symmetries[23]:

$$P : S^{ij}_{\alpha\beta}(x,y)[U] = \gamma^0_{\alpha\alpha'} S^{ij}_{\alpha'\beta'}(x^P, y^P)[U^P]\gamma^0_{\beta'\beta} \tag{51}$$

$$C : S^{ij}_{\alpha\beta}(x,y)[U] = (\gamma^0\gamma^2)_{\alpha\alpha'} S^{ji}_{\alpha'\beta'}(y,x)[U^C](\gamma^2\gamma^0)_{\beta'\beta} \tag{52}$$

$$T : S^{ij}_{\alpha\beta}(x,y)[U] = (\gamma^0\gamma^5)_{\alpha\alpha'} S^{ij}_{\alpha'\beta'}(x^T, y^T)[U^T](\gamma^5\gamma^0)_{\beta'\beta} \tag{53}$$

$$H : S^{ij}_{\alpha\beta}(x,y)[U] = \gamma^5_{\alpha\alpha'} S^{ji}_{\alpha'\beta'}(y,x)[U]\gamma^5_{\beta'\beta} \tag{54}$$

U^P, U^C, U^T are the parity reversed, charge conjugate, time reversed gauge configuration respectively.

B Basic syntax for MDP_QCD

- Classes defined in the file MDP_Lib1.h included by MDP_QCD.h:

```
class Toy_class;
class Matrix;
class Random_generator;
class JackBoot;
```

- Classes defined in the file MDP_QCD.h:

```
class gauge_field public: Toy_class<Complex, Nvol*Ndim*Nc*Nc>;
class pl_field    public: Toy_class<Complex, Nvol*6*Nc*Nc>;
class em_field    public: Toy_class<Complex, Nvol*6*Nc*Nc>;
class fermi_field public: Toy_class<Complex, Nvol*Nc*Nspin>;
class light_propagator;
```

  The elements of each field-type class can be accessed as matrices

```
Matrix gauge_field::operator() (site x, int mu);
Matrix em_field::operator()    (site x, int mu, int nu);
Matrix pl_field::operator()    (site x, int mu, int nu);
Matrix fermi_field::operator() (site x);
```

  or as complex numbers

```
Complex gauge_field::operator() (site x, int mu, int i, int j);
Complex em_field::operator()    (site x, int mu, int nu,
                                 int i,int j);
Complex pl_field::operator()    (site x, int mu, int nu,
                                 int i, int j);
Complex fermi_field::operator() (site x, int i, int alpha);
```

  To save memory em_field and pl_field only store those with components mu<nu. The class light propagator has peculiar class members and they are discussed in the proper section.

- Global functions:

```
site move(site x, int mu, int lmu);
site move(site x, int mu, int lmu, int nu, int lu);
site coordinate_space(x);
site position(int x0, int x1, int x2, int x3);
site position(int x0, site x_space);
float dist(site x, site y);
void start();
```

```
void stop();
Matrix plaquette(gauge_field U, site x, int mu, int nu);
Matrix staple_up(gauge_field U, site x, int mu, int nu);
Matrix staple_dw(gauge_field U, site x, int mu, int nu);
Matrix staple(gauge_field U, site x, int mu, int nu);
float average_plaquette(gauge_field U);
gauge_field cold();
gauge_field hot();
float  action(gauge_field U);
void   multihit(gauge_field U, int n_steps, n_hits);
pl_field compute_plaquettes(gauge_field U);
em_field compute_em_tensor(pl_field Up);
fermi_field mul_Q(gauge_field U, em_field T,
                  fermi_field psi, int dagger=1);
fermi_field mul_invQ(gauge_field U, em_field T,
                     fermi_field psi,
                     float delta=1e-4, int dagger=1);
fermi_field new_fermi_config(gauge_field U, em_field T);
fermi_field smearing(fermi_field psi, gauge_field U,
                     float epsilon, int level);
```

- Global functions for Input/Output:

```
void write(gauge_field U, char filename[], int gauge);
void  read(gauge_field U, char filename[], int gauge);
void write(fermi_field psi, char filename[], int gauge, int nfermi);
void  read(fermi_field psi, char filename[], int gauge, int nfermi);
void write(light_ptopagator S, char filename[], int gauge);
void  read(light_propagator S, gauge_field U, em_field T,
           char filename[], int gauge);
```

- Global constants (defined in `MDP_Settings.h`, included by `MDP_QCD.h`). They can be modified by the programmer.

	C++ with MDP_QCD	Value
lattice sites in time	Nx0	8
lattice sites is x_1	Nx1	4
lattice sites is x_2	Nx2	4
lattice sites is x_3	Nx3	4
n° colors	Nc	3
n° fermi configs	Nfermi	10

(55)

- Global constants that should not be modified by the programmer and can be used in `main` and in functions.

	C++ with MDP_QCD	Value
lattice sites in space	`Nspace`	`Nx1*Nx2*Nx3`
lattice sites in total	`Nvol`	`Nx0*Nspace`
n° dimensions	`Ndim`	`4`
n° spin components	`Nspin`	`4`
elements in adjoint	`Nadj`	`Nc*Nc-1`
	`DAGGER`	`-1`
new type	`site`	`long`
logical operator	`and`	`&&`
logical operator	`or`	`||`

(56)

- Global variables:

	C++ with MDP_QCD	type
β	`beta`	`float`
κ	`kappa`	`float`
c_{SW}	`c_SW`	`float`
$x+\widehat{\mu}$	`up[x][mu]`	`site`
$x-\widehat{\mu}$	`dw[x][mu]`	`site`
x_μ	`co[x][mu]`	`int`
$\gamma_{\mu=0..3}$	`Gamma[mu]`	`Matrix` 4×4
γ^5	`Gamma5`	`Matrix` 4×4
$\sigma_{\mu\nu}$	`Sigma[mu][nu]`	`Matrix` 4×4
$\sigma^{i=0..3}$ $(\sigma^0=1)$	`sigma[i]`	`Matrix` 2×2
$\lambda^{a=0..8}$ $(\lambda^0=1)$	`Lambda[a]`	`Matrix` 3×3
$\omega^{c=0..N_c^2}$ $(\omega^0=1)$	`Generator[c]`	`Matrix` $N_c\times N_c$

(57)

The variables β, κ and c_{SW} must be initialized in the main program before they are used. Note that the matrices $\omega^{c=1..N_c^2}$ form a general basis of Hermitian matrices for $SU(N_c)$. They coincide with the Pauli matrices for $N_c=2$, but not with the Gell-Mann matrices for $N_c=3$.

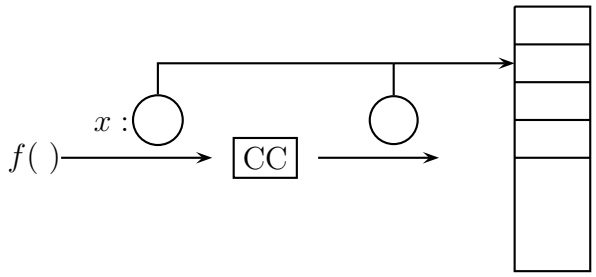


Figure 2: When the object x is returned the copy constructor (CC) is called and it creates a temporary object. The objects are represented as circles.

C `Toy_class` and how it works

Some books[1] about `C++` warn the reader about the problems of returning objects. In fact when an object is returned by a function, it is copied by the copy constructor into a temporary object. When the function terminates the temporary object is returned (fig. 2).

If the original object to be returned contains a dynamically allocated pointer (`p`) then its value is copied into the temporary object but the memory pointed by `p` is not. The problem arises when the function terminates because the destructor of the original object is called and the memory pointed to by `p` is deallocated (fig. 3). Hence the temporary object contains a pointer to a location of memory which is no longer allocated.

When this situation occurs, there is sometimes a runtime error such as "bus error" but often the program continues to run accessing to locations of memory containing random data without protection.

Consider for example the following code:

```
// THIS CODE IS WRONG!!!
template <class T, long imax> class Toy_class {
public:
   T *m;
   Toy_class() {
      m=new T[imax];
      for(i=0; i<imax; i++) m[i]=0;
   };
 };
 Toy_class f() {                  // function f()
    Toy_class<int,10> x;
    x.p[3]=5;
```

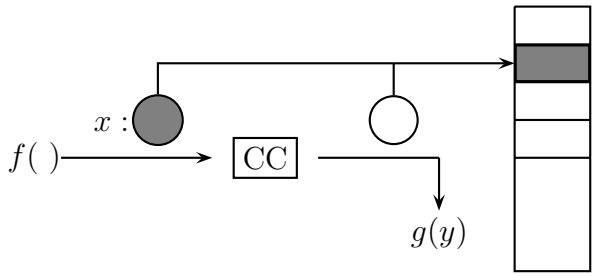


Figure 3: The temporary object created by the copy constructor (CC) is assigned to the argument of the calling function, then the original object x is destroyed and the memory is deallocated.

```
   return x;
};
T g(Toy_class<int,10> y) {       // function g()
   return y.p[3];
};
int main() {                     // main program
   cout << g(f());
   return 0;
};
```

It is supposed to print '5' on the screen, but it may give any random result: it is not safe!

There are two standard ways to overcome this problem:

- Redefine the copy constructor so that it will allocate new memory and will copy `p[i]` for every `i`.

  ```
  Toy_class (Toy_class &x) {
     if(p!=0) delete[] p;
     p=new T[imax];
     for(int i=0; i<imax; i++) p[i]=x.p[i];
  };
  ```

  This is pretty safe, but it may force you to copy a huge amount of memory when it is not necessary (fig. 4).

- Write all the functions so that their arguments are passed and returned by reference (so that the copy constructor is never called). Write the destructor

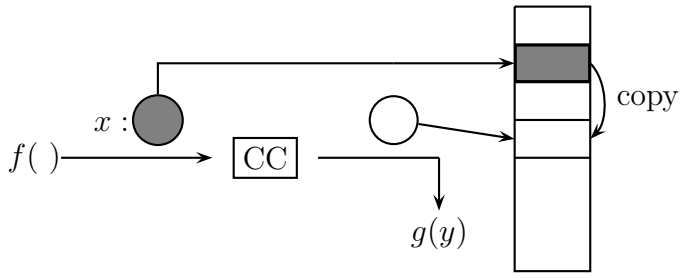


Figure 4: A possible solution: The memory is copied by the copy constructor before the destructor of x is called.

in such a way that, when an object is destroyed, the memory pointed to by `p` is not deallocated. It is also necessary to keep trace of the allocated memory and deallocate it somewhere in the program. This works, but only in very simple cases. This technique will become intractable in the presence of recursive functions which pass and return objects.

In this appendix a solution to this problem is proposed, based on the idea of attaching a `FLAG` to each object. The `FLAG` will contain information about the status of the object and it will be used to implement, in the most general framework, a simple and efficient way of passing and returning objects of any kind, even in recursive structures. Moreover all the tricks due to the dynamical allocations will be hidden within the basic methods of the class (constructor, copy constructor, destructor and assignment operator).

The word "efficient" here means that the program will automatically take care of the memory used and nothing will be copied if it is not necessary, hence the code will be optimized both in speed and memory usage. [10]

Suppose that one attaches a `FLAG`, i.e. a new member variable, to each object

```
enum value {F, T, C, H} FLAG;
```

and one uses it in the following way:

The `FLAG` is always set to (F)REE by the constructor. If the object is not going to be returned its `FLAG` will remain unchanged and the destructor will deallocate the memory pointed by its pointer when it is called. If the object is going to be returned its `FLAG` is changed to (R)ETURN and its destructor will

[10] The technique that will be explained is used to implement the class `Matrix`, `gauge_field`, `pl_field`, `em_field` and `fermi_field`.

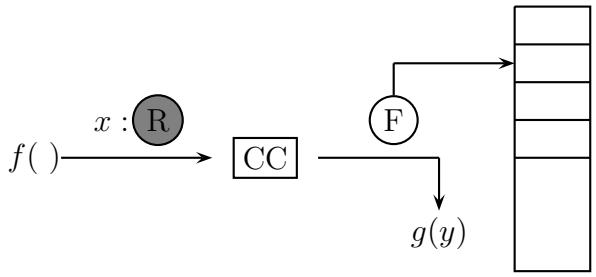


Figure 5: The destructor of x knows from the flag R that this object is going to be returned and it does not deallocate the memory. The temporary object y carries an F flag, hence it will be deallocated later.

not deallocate the memory in this case. On the other side the copy constructor that is going to create the temporary object will check the `FLAG` of its argument and if it is R, it will set the `FLAG` of the temporary object to F (fig. 5).

Moreover it is necessary to take care of the possibility of passing an F object as argument of a function: in this case the copy constructor will generate a temporary object with a C `FLAG` and the destructor will be implemented in such a way that the memory pointed by a C will never be deallocated.

In other words a F object is a normal object that sooner or later is going to be destroyed (in the same function, or method, where it has been created), while a C one is a copy. Its pointer is pointing to memory allocated by someone else (a F object existing at an higher level). On the other side an object R exists only in the brief instant between the moment when the copy constructor is called and when its own destructor is called. The memory that it is pointing to is not deallocated because the new F temporary object created by the copy constructor will contain a reference to it. Such a memory will be deallocated later when the destructor of the temporary object will be called somewhere automatically. The three possible situations are illustrated in fig. 6 (left).

To make the `FLAG` usage even safer it is better to define it as a private member and define a function (`prepare()`) that sets the `FLAG` of the object to R before it is returned.

A `FLAG` value (H)yperlink allows an object to contain a pointer to an address of memory allocated by an object of a different class. So that this memory will never be deallocated by the destuctor of the object.

According with these prescriptions the program class Toy_class should be defined in the following way:

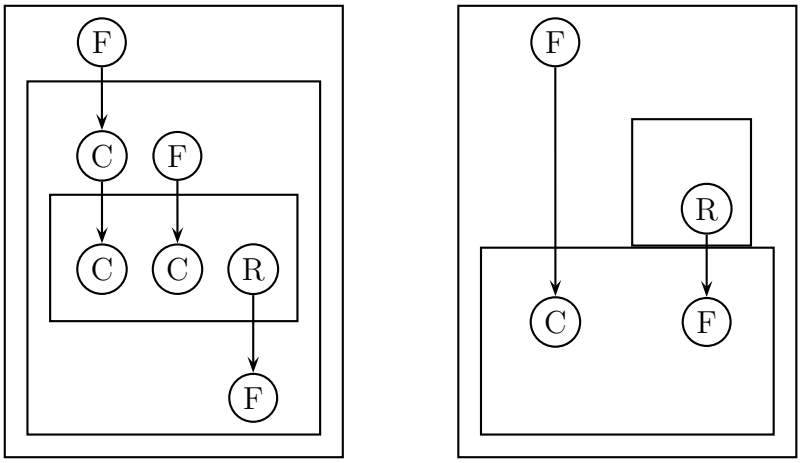


Figure 6: Different kinds of objects that may be passed to (and returned by) a function are represented. In these diagrams the copy constructor is automatically called when the arrow crosses the boundary of a box (representing a functional level in the program).

```
template <class T, long imax>
class Toy_class {
public:
  enum value {F,R,C,H} FLAG;
  T *m;
  Toy_class() {
    register long i;
    FLAG=F;
    m=new T[imax];
    for(i=0; i<imax; i++) m[i]=0;
  };
  Toy_class(T *m0) {
    FLAG=H;
    m=m0;
  };
  Toy_class(const Toy_class &a) {
    m=a.m;
    switch(a.FLAG) {
    case F: FLAG=C; break;
    case R: FLAG=F; break;
    case C: FLAG=C; break;
    case H: FLAG=H; break;
    };
```

```
  };
  ~Toy_class() {
    if((FLAG==F) && (m!=0)) delete[] m;
  };
  operator= (Toy_class a) {
    register long i;
    switch(FLAG) {
    case H:
      for(i=0; i<imax; i++) m[i]=a.m[i];
      break;
    case F:
      if(a.FLAG==F) {
        if ((m!=0) && (m!=a.m)) delete[] m;
        m=a.m;
        a.m=0;
      } else {
        for(i=0; i<imax; i++) m[i]=a.m[i];
      };
      break;
    case C:
      if(a.FLAG==F) {
        m=a.m;
        a.m=0;;
      } else {
        m=new T[imax];
        for(i=0; i<imax; i++) m[i]=a.m[i];
      };
      FLAG=F;
      break;
    case R:
      error("What the Hell are you doing?");
      break;
    };
  };
  inline friend void prepare (Toy_class &a) {
    register long i;
    T* m;
    switch(a.FLAG) {
    case F: a.FLAG=R; break;
    case R:
      printf("You shoud not call prepare(); twice\n");
      break;
    default:
      m=new T[imax];
      for(i=0; i<imax; i++) m[i]=a.m[i];
      a.m=m;
      a.FLAG=R;
      break;
    };
  };
};
```

Now the following program works!

```
Toy_class f() {               // function f()
   Toy_class<int,10> x;
   x.p[3]=5;
   return x;
};
T g(Toy_class<int,10> y) {    // function g()
   return y.p[3];
};
int main() {                  // main program
   cout << g(f());
   return 0;
};
```

Note how the information contained in the `FLAG` has been used by the functions acting on the objects (in particular by the assignment operator). To optimize both speed and memory usage it is required for a C object (i.e. the copy made by the copy constructor of an F object existing at a higher level, see fig. 6 (right)) to be copied element by element into a new location of the memory, otherwise only its pointer is copied (the fastest way). Moreover when a new value is assigned to a C type object it is promoted to type F.
As an exercise the reader is suggested to work out the deatils.
Once the class with the `FLAG` has been implemented then all these technicalities can be forgotten but the safety rules, stated in the next appedinx, should always be kept in mind!

D Safety rules

Since the copy constructor has been redefined and it is now `FLAG` dependent there are a few safety rules to follow to be sure that everything is working properly. These safety rules applys to objects of class:

```
Toy_class
Matrix
gauge_field
pl_field
em_field
fermi_field
```

These safety rules are:

★ **Always, before returning an** *object*, **change its** `FLAG` **to R, i.e. call** `prepare(`*object*`);`

★ **Never explicitly use the copy constructor.**

★ **Do not call the assignment operator of the argument of a function within the function itself (unless you understood completely how `Toy_class` works).**

Attention: Different machines may have a different internal representations of real numbers. Therefore some care must be taken when copying data between different machines. This also holds for the file containing the seed:
`RandomBuffer.seed`
Therefore:

★ **Do not copy the seed from one machine to another, let the `start();` instruction create a new seed file.**

Acknowledgemnts

It is a pleasure to acknowledge C. Sachrajda, L. Del Debbio, J. Flynn, V. Lesk, J. Generowicz, K. Anderson (Southampton University) for useful discussions and suggestions. I also wish to thank C. Rebbi (Boston University) and C. Michael (Liverpool University) for letting me study their FORTRAN montecarlo codes, from which I learned a lot. The random number generator for real numbers encoded in `MDP_Lib1.h` has been written by M. Beccaria and L. Del Debbio.

References

[1] G. Satir and D. Brown, C++ The Core Language, O'Reilly and Associates (1995)

[2] B. Stroustup, C++ Programming Language, Addison-Welsey (1997)

[3] K. G. Wilson, Phys. rev. **D10** (1974) 2445

[4] K. Symanzik, Nucl. Phys. **B226** (1983) 187,205

[5] B. Sheikoleslami and R. Wolhert, Nucl. Phys. **B259** (1985) 572

[6] M. Creutz, Quarks, Gluons and Lattices, Cambridge University Press (1983)

[7] Rothe, Lattice Gauge Theories, World Scientific

[8] I. Montvay and G. Münster, Quantum Fields on a Lattice, Cambridge

[9] M. Di Pierro and C. T. Sachrajda, A Lattice Study of Spectator Effects in Inclusive Decays of B-Mesons, Nucl. Phys. **B534** (1998); hep-lat/9805028

[10] G. De Divitiis, L. Del Debbio, M. Di Pierro, J. Flynn, C. Michael and J. Peisa, Towards a lattice determination of the $B^*B\pi$ coupling, accepted for publication on JHEP (1998); hep-lat/9807032

[11] M. Di Pierro, Spectator Effects in Inclusive Decays of Beauty Hadrons, to apper in the proceedings of Lattice98 (1998); hep-lat/9809083

[12] M. Di Pierro and C. T. Sachrajda, Spectator Effects in Inclusive Decays of Λ_b from Lattice Simulations (to be submitted).

[13] N. Cabibbo and E. Marinari, A new method for updating $SU(N)$ matrices in computer simulations of gauge theories, Phys. Lett. **119B** (1982) 387

[14] J. Shao and D. Tu, The Jackknife and Bootstrap, Springer Verlag (1995)

[15] M. Creutz, Quantum Fields on the Computer, World Scientific (1992)

[16] C. T. Sachrajda, Lattice Simulations and Effective Theories, Lectures presented at the Advanced School on Effective Theories, Almunecar (Spain) June 1995; hep-lat/960527

[17] Heatlie et al., Nucl. Phys. **B352** (1991) 266

[18] G. Parisi, R. Petronzio and C. Rapuano, Phys. Lett. **B128** (1983) 418

[19] C. Michael and J. Peisa, Maximal variance reduction for stochastic propagators with applications to the static quark spectrum, Phys.Rev. **D58** (1998)

[20] S. Güsken *et al.*, Phys. Lett. B227 (1989) 266

[21] G. Banhot, The Metropolis Algorithm, Rep. Prog. Phys. 51 (1988) 429

[22] M. Luscher, Advanced Lattice QCD, Talk given at Les Houches Summer School in Theoretical Physics, Session 68, (1998); hep-lat/9802029

[23] C. Bernard in "From Action to Answers", Proceedings of the 1989 TASI in Elementary Particle Physics, Boulder, World Scientific